SOUTENANCE PFE · VERMEG

Assistant DevSecOps intelligent Questions possibles du jury et réponses

47 questions classées par thème, avec des réponses courtes à dire avec vos mots.

Sommaire

01	Contexte et problématique	5 questions
02	Solution et besoins	5 questions
03	Conception et architecture	6 questions
04	Intelligence artificielle et machine learning	10 questions
05	Sécurité	6 questions
06	Réalisation, méthode et tests	5 questions
07	Démonstration : questions pièges	4 questions
08	Limites, perspectives et bilan	6 questions

Comment répondre

- Répondez d'abord en une phrase, puis détaillez si le jury relance.
- Reformulez une question floue avant d'y répondre : « Si je comprends bien, vous demandez… ».
- Ne connaissez pas la réponse ? Dites-le, puis expliquez comment vous la trouveriez.
- Reliez chaque réponse technique à un bénéfice : gain de temps, sécurité, qualité.
- Gardez sous la main : la présentation, l'application lancée, un fichier de samples/ à importer.

Les chiffres cités (seuils, nombre de tests, formules de score) proviennent du code du projet. Les réponses marquées « réponse personnelle » sont à adapter à votre expérience.

01 Contexte et problématique

Q1 Pouvez-vous résumer votre projet en une phrase ?

C'est un assistant qui réunit, au même endroit, les logs, les résultats de tests et les failles de sécurité d'une application, les analyse automatiquement et utilise l'intelligence artificielle pour expliquer chaque incident et proposer une correction, que l'équipe valide avant d'agir.

Conseil Apprenez cette phrase par cœur : c'est souvent la première question.

Q2 Quel problème concret résolvez-vous chez Vermeg ?

Aujourd'hui, comprendre un incident oblige à ouvrir plusieurs outils un par un, à lire les logs à la main et à compter sur l'expérience de quelques experts. Le diagnostic est lent et les alertes ne sont pas priorisées. L'assistant centralise ces signaux, les relie entre eux et propose une explication.

Q3 Pourquoi c'est important dans le secteur de Vermeg ?

Vermeg édite des logiciels pour la banque, l'assurance et la gestion d'actifs. Dans la finance, une panne ou une faille coûte cher et touche la confiance des clients : livrer plus vite sans perdre en qualité ni en sécurité est un enjeu direct.

Q4 Qu'est-ce que le DevSecOps, en termes simples ?

C'est intégrer la sécurité à chaque étape du développement et de la livraison, au lieu de la vérifier seulement à la fin. Mon assistant en est un outil : il surveille la qualité et la sécurité en continu, à chaque livraison.

Q5 Existe-t-il déjà des solutions sur le marché ? Pourquoi ne pas les utiliser ?

Oui, chaque outil existe séparément : SonarQube pour le code, Trivy et OWASP pour les dépendances, Grafana/Loki pour les logs, et des plateformes commerciales d'observabilité. Leur limite : ils ne sont pas reliés entre eux, et une solution commerciale complète est coûteuse. Mon apport est de les réunir, de corréler les données (un test en échec avec les logs du même moment) et d'ajouter une explication par IA, sur une base open source.

Conseil Ne dénigrez pas les outils existants : présentez votre projet comme une couche qui les rassemble.

---

02 Solution et besoins

Q6 Quelles sont les grandes fonctionnalités ?

Quatre briques : les logs (import, regroupement des erreurs, détection de pics), les tests (historique, tests instables, lien avec les logs), la sécurité (scans, score de priorité de 0 à 100, suivi des corrections) et l'IA (explication d'incident, recommandations à valider, copilote conversationnel). Plus un tableau de bord avec un score de santé et une quality gate.

Q7 Qui sont les utilisateurs et que peut faire chacun ?

Le développeur consulte et analyse, et utilise le copilote. Le responsable qualité peut en plus gérer les failles, valider les recommandations de l'IA et consulter le journal d'audit. L'administrateur gère en plus les comptes. Enfin, la chaîne CI/CD est un acteur technique qui publie ses résultats avec une clé d'API.

Q8 Quels sont vos besoins non fonctionnels et comment les avez-vous respectés ?

Sécurité : authentification JWT, rôles, mots de passe hachés (BCrypt), verrouillage après 5 échecs. Disponibilité : si l'IA en ligne tombe, des modèles de secours puis un moteur local prennent le relais. Performance : les tâches longues (scans, tests) tournent en arrière-plan. Maintenabilité : trois services indépendants et 72 tests automatisés. Portabilité : tout se déploie avec Docker Compose.

Q9 Qu'est-ce que la quality gate ?

C'est le « feu vert » de livraison. Elle échoue si une condition n'est pas respectée : plus de 0 vulnérabilité critique, plus de 5 hautes, moins de 95 % de tests réussis, moins de 40 % de couverture, ou plus de 10 anomalies critiques. Le pipeline CI l'interroge et s'arrête si elle répond FAILED.

Conseil Les seuils sont configurables dans application.yml : dites-le si on vous demande pourquoi ces valeurs.

Q10 Comment est calculé le score de santé ?

On part de 100 et on retire des points plafonnés par brique : vulnérabilités critiques et hautes (jusqu'à 40 points), taux d'échec des tests (jusqu'à 25), anomalies critiques (jusqu'à 20) et pics d'erreurs (jusqu'à 10). Au-dessus de 80 c'est « Bon », puis « À surveiller », « Dégradé » et « Critique » sous 40.

03 Conception et architecture

Q11 Expliquez votre architecture logique.

Trois couches. La présentation : l'application web Angular. Le métier : le backend Spring Boot, où chaque requête passe d'abord par la sécurité puis par un des cinq modules (logs, tests, sécurité, IA, pilotage). Les données et services : PostgreSQL, le service IA en Python et les outils externes (SonarQube, Trivy, OWASP, Loki).

Q12 Et l'architecture physique ?

Tout tourne dans des conteneurs Docker sur un même serveur : frontend (nginx), backend, base de données et service IA ; SonarQube et Loki/Grafana sont optionnels. L'utilisateur passe par le navigateur en HTTPS, le pipeline CI envoie ses résultats avec une clé d'API, et seul le service IA communique avec l'extérieur, pour joindre le modèle de langage chez Groq.

Q13 Pourquoi avoir séparé le service IA dans un microservice Python ?

Pour trois raisons : l'écosystème de l'IA et du machine learning est en Python (scikit-learn, SDK Groq) ; un problème côté IA ne bloque pas le reste de l'application ; et on peut changer de fournisseur de modèle sans toucher au backend, qui ne connaît qu'un contrat JSON.

Q14 Pourquoi ces choix technologiques (Angular, Spring Boot, PostgreSQL) ?

Ce sont des technologies d'entreprise, robustes et très répandues. Spring Boot apporte la sécurité (Spring Security), l'accès aux données (JPA) et une intégration facile avec Maven et JUnit. Angular est structuré et typé avec TypeScript, adapté aux applications métier. PostgreSQL est une base relationnelle fiable et libre.

> **Conseil** Si vous ne connaissez pas la stack exacte de Vermeg, ne l'inventez pas : restez sur les qualités de chaque outil.

Q15 Pourquoi une base relationnelle et pas NoSQL ?

Les données sont structurées (utilisateurs, vulnérabilités avec statuts, exécutions de tests, recommandations) et on a besoin de filtres, de tris et de cohérence. Une base relationnelle avec des index sur les colonnes recherchées (date, niveau, source, statut) répond bien à ce besoin.

Q16 Présentez votre diagramme de classes.

L'utilisateur est au centre : chacune de ses actions crée un événement d'audit et c'est lui qui valide les recommandations IA. Une recommandation explique un incident : un log, une exécution de tests ou une vulnérabilité. Les exécutions de tests sont corrélées aux logs produits pendant la même période.

---

04 Intelligence artificielle et machine learning

Q17 Quelle IA utilisez-vous exactement ?

Deux types. Du machine learning classique avec scikit-learn : TF-IDF + DBSCAN pour regrouper les erreurs, et Isolation Forest pour détecter les moments anormaux. Et un grand modèle de langage (LLM), gpt-oss-120b via Groq, pour expliquer les incidents et répondre aux questions dans le copilote.

Q18 Comment fonctionne le regroupement des erreurs ?

D'abord on normalise les messages : les nombres, identifiants, adresses IP et chemins sont remplacés par des marqueurs. Ensuite TF-IDF transforme chaque message en vecteur selon les mots importants, et DBSCAN regroupe les messages proches (distance cosinus). Des milliers de lignes deviennent quelques familles d'erreurs.

Q19 Pourquoi DBSCAN et pas K-means ?

K-means demande de connaître à l'avance le nombre de groupes, ce qui est impossible avec des logs. DBSCAN trouve lui-même le nombre de groupes, accepte des groupes de formes variées et isole le bruit (les messages uniques) au lieu de les forcer dans un groupe.

Conseil Question classique : préparez-la bien.

Q20 Comment détectez-vous les pics d'erreurs ?

On compte les erreurs de chaque service par fenêtre de 5 minutes. Une fenêtre est un pic si elle s'écarte fortement de l'habituel, mesuré par un z-score robuste (médiane et écart absolu médian, peu sensibles aux valeurs extrêmes, seuil 2,5). En complément, le modèle Isolation Forest isole les fenêtres atypiques.

Q21 Pourquoi Isolation Forest ?

C'est un algorithme non supervisé : il n'a pas besoin d'exemples étiquetés « normal / anormal », que nous n'avons pas. Il isole les points atypiques en peu de coupures aléatoires, ce qui est rapide et adapté à des séries temporelles simples.

Q22 Comment évitez-vous que le modèle de langage invente des réponses (hallucinations) ?

Plusieurs garde-fous. Le copilote reçoit un instantané réel du projet et a pour consigne de ne s'appuyer que sur lui, et de dire quand l'information manque. Les explications sont demandées dans un format JSON strict. Surtout, aucune proposition n'est appliquée automatiquement : un humain valide ou rejette chaque recommandation, et c'est tracé.

Conseil Mot-clé à placer : « l'humain garde la main ».

Q23 Que se passe-t-il si Groq est indisponible ?

Il y a une cascade : si le modèle principal échoue (quota, panne, réponse vide), l'assistant essaie deux modèles de secours (llama-3.3-70b puis llama-3.1-8b). Si tout échoue, un moteur local à base de règles répond : il reconnaît 13 types d'incidents connus et répond aux questions courantes. L'assistant reste donc utile hors ligne.

Q24 Les données de Vermeg partent-elles dans le cloud ? Est-ce un problème de confidentialité ?

Seul le service IA communique avec Groq, et il n'envoie qu'un résumé : messages d'erreur, noms de composants, statistiques. Le code source n'est pas envoyé. Pour un usage en production chez Vermeg, on pourrait héberger un modèle open source en interne, puisque le backend ne dépend que d'un contrat JSON : c'est une perspective naturelle.

Conseil Question sensible : reconnaissez la limite plutôt que de la nier.

Q25 Comment le copilote connaît-il l'état du projet ?

À chaque question, le backend construit un instantané à jour : score de santé, quality gate, statistiques de logs, anomalies, tests, tests instables, vulnérabilités, recommandations, activité récente. Cet instantané est ajouté au prompt avec les derniers échanges de la conversation. Le modèle répond donc sur les vraies données du moment.

Q26 Comment avez-vous évalué la qualité des réponses de l'IA ?

Par des tests automatisés sur des incidents connus (le moteur de règles doit reconnaître, par exemple, un pool de connexions saturé ou Log4Shell), par des essais manuels sur des scénarios réalistes, et par le circuit de validation : le nombre de recommandations validées ou rejetées est lui-même un indicateur de qualité suivi dans le tableau de bord.

Conseil N'avancez pas de pourcentage de précision que vous n'avez pas mesuré.

---

05 Sécurité

Q27 Comment sécurisez-vous l'application elle-même ?

Authentification par jeton JWT sans session, rôles vérifiés sur chaque route et chaque action sensible, mots de passe hachés avec BCrypt, verrouillage du compte 5 minutes après 5 échecs, clé d'API dédiée au pipeline comparée en temps constant, secrets dans des variables d'environnement, conteneurs non root, et journal d'audit des actions sensibles.

Q28 Pourquoi JWT ?

Le jeton est autonome et signé : le serveur n'a pas besoin de garder de session, ce qui simplifie le passage à plusieurs instances. Il expire après 8 heures, et le frontend déconnecte l'utilisateur dès que le serveur répond 401.

Q29 Comment calculez-vous le score de priorité d'une vulnérabilité ?

Sur 100 : la gravité (score CVSS ou niveau de sévérité) compte jusqu'à 80 points ; +5 si la faille vient d'une dépendance ou d'une image réellement déployée ; +8 si une version corrective existe, car on peut agir tout de suite ; et jusqu'à +7 selon l'ancienneté. On traite donc d'abord ce qui est grave et facile à corriger.

Q30 Différence entre SonarQube, OWASP Dependency-Check et Trivy ?

SonarQube analyse notre propre code source. OWASP Dependency-Check analyse les bibliothèques Java que nous utilisons. Trivy analyse les images Docker, y compris les paquets du système. Les trois sont complémentaires, et l'assistant fusionne leurs résultats.

Q31 Qu'est-ce qu'une CVE et un score CVSS ?

Une CVE est l'identifiant public d'une faille connue, par exemple CVE-2021-44228 pour Log4Shell. Le CVSS est une note de 0 à 10 qui mesure sa gravité technique.

Q32 Que se passe-t-il quand une faille est corrigée ?

À chaque nouveau scan, l'assistant compare les résultats : une faille qui n'apparaît plus passe automatiquement en « corrigée », et une faille qui revient est marquée « réouverte ». Cela permet aussi de mesurer le délai moyen de correction.

---

06 Réalisation, méthode et tests

Q33 Quelle méthode de travail avez-vous suivie ?

La méthode agile Scrum, en six sprints de deux à trois semaines : fondations et authentification, logs, tests et corrélation, sécurité, intelligence artificielle, puis tableau de bord, intégration continue et déploiement. Chaque sprint livrait une fonctionnalité testée, présentée puis améliorée grâce aux retours.

Q34 Comment avez-vous testé votre application ?

72 tests automatisés : 45 pour le backend (dont 12 tests d'intégration de la sécurité de l'API), 24 pour le service IA et 3 pour le frontend. Ils tournent automatiquement dans le pipeline CI à chaque modification. Les tests backend utilisent une base en mémoire, sans dépendance externe.

Q35 Comment l'assistant s'intègre-t-il dans la CI/CD ?

Des pipelines sont fournis pour GitHub Actions, GitLab CI et Jenkins. Ils lancent les tests et les scans, puis un script envoie les rapports à l'assistant avec une clé d'API et interroge la quality gate. Si elle répond FAILED, le pipeline s'arrête et la version n'est pas livrée.

Q36 Comment relier un test en échec aux logs ?

L'assistant mémorise la période pendant laquelle les tests ont tourné. Pour chaque échec, il cherche les logs émis pendant cette période, puis classe les plus pertinents par ressemblance avec le message d'erreur. Le développeur voit directement les logs qui expliquent probablement l'échec.

Q37 Quelle a été la plus grande difficulté technique ?

À adapter à votre vécu. Exemples crédibles : rendre l'IA fiable malgré les quotas du service en ligne (d'où la cascade de modèles et le moteur local), lire des formats de logs et de rapports très variés, ou corréler correctement tests et logs dans le temps.

Conseil Racontez une vraie difficulté, ce que vous avez essayé et ce que vous en retenez.

07 Démonstration : questions pièges

Q38 La démo utilise des données fictives : est-ce représentatif ?

Les données de démonstration reproduisent des incidents réels typiques (pool de connexions saturé, deadlock, failles connues comme Log4Shell) et des rapports produits par les vrais outils. La plateforme accepte aussi de vrais fichiers : on peut importer un log ou un rapport en direct.

Conseil Ayez un fichier de samples/ prêt à importer si on vous le demande.

Q39 Pourquoi l'IA a-t-elle répondu en « mode heuristique » pendant la démo ?

Cela signifie que le modèle en ligne n'était pas disponible à cet instant (quota ou réseau) : le plan B a pris le relais, comme prévu. C'est justement une exigence de disponibilité du projet.

Q40 Si je vous dis « bonjour » ou une question hors sujet, que fait le copilote ?

Il répond poliment en une phrase et propose son aide, sans afficher de données. Pour une question sur le projet, il ne répond qu'à ce qui est demandé ; s'il n'a pas l'information, il le dit et indique l'écran où la trouver.

Q41 La conversation est-elle conservée ?

Oui : elle reste affichée quand on change d'écran et elle est retrouvée après une déconnexion ou un rechargement de page, séparément pour chaque utilisateur. Elle est stockée dans le navigateur ; la stocker en base est une évolution prévue.

---

08 Limites, perspectives et bilan

Q42 Quelles sont les limites de votre solution ?

Le déploiement vise un seul serveur, sans haute disponibilité. Le copilote ne reçoit qu'un résumé limité du projet. L'historique de conversation est stocké dans le navigateur. Les quotas gratuits de Groq limitent le nombre de requêtes. Et l'IA ne voit pas le code source, ses propositions restent des pistes à valider.

Conseil Citer ses limites montre de la maturité : ne les cachez pas.

Q43 Quelles perspectives ?

Quatre étapes : passer à l'échelle avec Kubernetes ; nourrir l'IA de la documentation interne de Vermeg (approche RAG) ; proposer les correctifs directement sous forme de merge request ; et à terme, prédire les incidents avant qu'ils ne surviennent.

Q44 Qu'est-ce que le RAG ?

Retrieval-Augmented Generation : avant de répondre, on recherche dans une base documentaire (par exemple la documentation interne) les passages utiles, et on les donne au modèle. Il répond ainsi avec des informations propres à l'entreprise, sans réentraînement.

Q45 Quelle valeur apporte votre projet à l'entreprise ?

Un gain de temps sur le diagnostic, puisqu'on passe d'une enquête manuelle outil par outil à une vue unique avec une explication. Une meilleure priorisation des failles. Un savoir moins dépendant de quelques experts. Et une livraison mieux contrôlée grâce à la quality gate. Des indicateurs comme le délai moyen de correction ou le taux de validation des recommandations permettent de le suivre.

Conseil Ne donnez pas de gain chiffré que vous n'avez pas mesuré : parlez d'indicateurs suivis.

Q46 Qu'avez-vous appris pendant ce stage ?

Réponse personnelle. Pistes : concevoir une architecture complète de bout en bout, sécuriser une API, intégrer du machine learning et un modèle de langage dans un vrai produit, travailler en Scrum avec des livraisons régulières, et expliquer un sujet technique à un public varié.

Q47 Si c'était à refaire, que changeriez-vous ?

Exemple de réponse : définir plus tôt des indicateurs pour mesurer objectivement la qualité des réponses de l'IA, et prévoir dès le début un modèle hébergé en interne pour la confidentialité.